

UNIVERSITY OF LONDON

INTERNATIONAL PROGRAMMES

BSc Computer Science and Related Subjects



CM3070 PROJECT

PRELIMINARY PROJECT REPORT

<Financial Advisor Bot>

Author: Puriwat Puriyapan

Student Number : UOL 240662321 / SIM 10241944

Date of Submission: 28 June 2026

Supervisor :Mr Gerald Chua

<NOTE that the overall maximum word count for the PPR is 6000 words excluding Appendices and References. If you follow strictly the maximum word count for each chapter, it will exceed 6000 words, i.e. you will need to decide which chapter(s) to trim to meet the overall word count limit>

Contents

CHAPTER 1:	INTRODUCTION	3 - 6
CHAPTER 2:	LITERATURE REVIEW	7 - 12
CHAPTER 3:	PROJECT DESIGN	13 - 18
CHAPTER 4:	FEATURE PROTOTYPE	19 - 23
CHAPTER 5:	REFERENCES	24 - 25

CHAPTER 1:

INTRODUCTION

The problem / motivation:

Young adults nowadays between 18 and 30 years old want to start investing in stocks to make their own money. However, they are facing a real barrier to doing it well. Professional financial advice is expensive, and first-time investors lack both the funds to pay for an advisor and the financial literacy to interpret market data on their own. Instead, they rely on free stock apps, social media tips, and online forums sources that either overwhelm them with jargon or hand them BUY and SELL signals with no reasoning behind them.

The problem is not lack of information about investing. It is a lack of understandable information at the moment a decision needs to be made. A new investor opening a typical stock app sees prices, charts, and sometimes a recommendation, but no explanation of why one stock looks promising to buy and another does not.

A free, easy-to-use tool that shows stock recommendations in plain English, with the reasoning made explicit, addresses this gap. By telling the user not only what to consider buying or selling but why, the system allows them to decide whether to trust each recommendation, learn something about investing each time they use it, disagree with the model when their own view differs, and feel in control of their money rather than outsourcing every decision.

(220 words)

Project Concept:

The project is to develop AI that lets users input the company they want to buy stock in, analyse price data by using historical data and returns a BUY/SELL recommendation with current price in plain English on a web interface. This system is designed for young adults between 18-30 years old who want to start investing in stock but can't afford professional advice. The system helps them have the first step of investing because this application will help to make an easier decision whether to buy or sell on a stock they are interested in. This addresses the gap identified above by explaining every recommendation in plain English and avoiding technical jargon.

The system works as follows. When a user enters a ticker on the web interface, the system fetches historical data. The system computes technical indicators from the price data. These indicators are then passed to a trained model Random Forest as a baseline and Long Short-Term Memory (LSTM) as the main model which predicts BUY or SELL. Next, SHAP values identify which indicators drove the prediction. These outputs are then translated into plain English and displayed on the web interface alongside the recommendation and a chart.

This approach, on the other hand, closes this gap by offering explanations in simple English so that people may comprehend the recommendation without having to grasp financial jargon. Additionally, it helps non-technical users by eliminating the need for coding knowledge and offering a free web interface that displays recommendations, explanations, and charts.

(251 words)

Project Template:

Project Idea 4.2: Financial Advisor Bot from the CM3020 Artificial Intelligence course was chosen as the project template for this project. By using machine learning techniques for stock data, offering a web interface for non-technical users, outlining recommendations for novice investors, and assessing the system through backtesting, this project satisfies the criteria of that brief.

(56 words)

Aim, Objectives, and Deliverables:

This project aims to create a financial adviser bot for young, non-technical investors using a web interface. Recommendations and explanations are shown by this method in simple English.

(28 words)

Deliverable 1 Data pipeline:

The data pipeline's goal is to retrieve price information from Yfinance. This offers daily historical price data for any US stock. The ML models then analyse this data.

(28 words)

Deliverable 2 Technical indicators:

The technical indicators' goal is to convert unprocessed pricing data into characteristics that the model can use to identify trends. A module that computes indicators like MA, RSI, and MACD is the deliverable.

(33 words)

Deliverable 3 Machine learning models:

Using Random Forest as a baseline and LSTM as the primary model, the machine learning models aim to forecast BUY/SELL from the technical indications. A trained RF model and a trained LSTM model are the deliverables, which may be compared to see whether LSTM improves upon the baseline.

(49 words)

Deliverable 4 Backtesting / evaluation:

Backtesting aims to assess the trained models' performance on data that they were not exposed to during training. The deliverable is a backtesting module that evaluates model performance against a buy-and-hold baseline using walk-forward validation.

(35 words)

Deliverable 5 Explainability:

The objective of the explainability layer is to identify how much each technical indicator contributed to a given prediction. The deliverable is a SHAP analysis module and a plain-English generator that converts SHAP values into readable explanations.

(37 words)

Deliverable 6 Web interface:

The web interface's goal is to enable non-technical users to access the system. The deliverable is a Flask-built website that displays a chart, explanation, and recommendation when a user enters a ticker.

When combined, these six deliverables provide a functional financial adviser bot that can provide youthful, non-technical investors with suggestions and explanations in simple English.

(56 words)

(Total: 793 words)

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

Plainstock is an advisor bot that recommends, explains, and graphs stock advice in plain English for non-technical and new investors. It solves issues with current apps, which are costly, display raw data without suggestions, or depend on human advisors whose counsel may be sentimental. Four topics are covered in this chapter: explainable AI (SHAP) for plain-English explanations; technical indicators utilised as features; machine learning techniques for prediction (LSTM and Random Forest); and current stock apps and their limitations. By offering users a free tool that offers suggestions and explanations in simple English along with graphs shown on a website, the project seeks to close this gap.

(106 words)

2.2 Existing Apps and Their Limitations

Several platforms already in use attempt to assist retail investors, but each has drawbacks that leave a gap for young investors and non-technical users. Users can purchase and sell stocks, ETFs, and other assets on their phones using the commission-free trading software Robinhood (Robinhood, 2026). It makes investing easier, but it only provides pricing and charts, not an explanation of why or not a stock is a suitable option. Additionally, it has been accused of "gamifying" investment; in response to claims that its design encouraged reckless trading among novice users, it eliminated a confetti animation (Money, 2021). This illustrates the issue with a tool that produces results but lacks logic.

At the other end is Bloomberg Terminal. It offers vast amounts of expert financial data and analysis, but it is too costly for the average youthful investor, costing roughly \$31,980 annually for a single user in 2026 (Godel Discount, 2026) and requiring training due to its heavy vocabulary and specific commands. Open-source GitHub trading bots are free, but they are not appropriate for non-technical consumers because they require coding knowledge to set up and produce numbers rather than plain English. Personalised advice is provided by human financial advisors, but it is costly, delayed, and not available on demand for a single stock. It can also be influenced by emotion or commission.

Plainstock addresses the need left by none of these tools by offering a free, straightforward explanation of a single stock suggestion to a non-technical user.

(247 words)

2.3 Machine Learning Methods for Stock Prediction

A supervised ensemble learning approach called Random Forest (Breiman, 2001) creates a large number of individual decision trees during training and delivers the mean prediction for regression or the mode of the classes for classification. It was selected as the baseline model because it outperforms a naive model by handling mixed data types well and predicting whether a stock is likely to increase or decrease by capturing complicated, non-linear market interactions rather than just repeating the previous price.

LSTM (Long Short-Term Memory) is a neural network that is good at learning from sequential, time-ordered data (Hochreiter and Schmidhuber, 1997). It is suitable for this project because stock data is sequential. Each trading day produces a new set of values and LSTM can retain important information over long time periods. Fischer and Krauss (2018) applied LSTM to S&P 500 stocks from 1992 to 2015 to predict next-day direction, and it beat the other models they tested. However, the reported returns were strongest before transaction costs and before 2010, suggesting that the strategy's practical profitability may not persist in more recent markets.

The inclusion of indicators like RSI and MACD as input features in this study is supported by Nelson et al. (2017), who combined LSTM with technical indicators and achieved better results than various traditional machine learning techniques. The results, however, might not apply to all stocks, eras, or market conditions because their analysis was conducted on a small number of markets and historical datasets.

In this project, the LSTM model achieved an accuracy of 0.552, beating Random Forest at 0.464, but neither model beat the majority baseline of 0.611. This is related to Fischer and Krauss (2018), who found that the benefit of LSTM prediction faded after 2010. My result continues this finding, showing that the predictive edge is hard to find in recent market data.

(310 words)

2.4 Technical Indicators as Features

Technical indicators are figures that summarise a stock's performance based on historical price and volume data. Four primary indicators are used by Plainstock as model input characteristics. When the current price is higher than the moving average, the recent trend is upward. The Moving Average (MA) is the average price over the previous N days; it smoothes out daily noise and displays the trend. Introduced by Wilder in 1978, the Relative Strength Index (RSI) is a number between 0 and 100 that indicates how quickly and significantly a price has increased or decreased; a value above 70 indicates that the stock is overbought, while a value below 30 indicates that it is oversold, capturing momentum.

Appel (2005) created MACD (Moving Average Convergence Divergence), which compares a short-term and a longer-term moving average. When the short-term crosses above the long-term, it may indicate the beginning of upward momentum. Bollinger Bands, developed by Bollinger (2001), indicate whether a price is unusually high or low compared to its recent average by drawing a band above and below the moving average depending on volatility. Instead of using raw prices as features, these indicators capture trends, momentum, and volatility in a way that makes it easier for the model to learn. Their usage in this research is supported by Nelson et al. (2017), who demonstrated that inputting technical indications into an LSTM gave superior results than various conventional methods.

(234 words)

2.5 Explainable AI (SHAP)

A young investor cannot determine whether an advice to purchase or sell should be believed because machine learning models like Random Forest and LSTM do not provide an explanation for their predictions. Lundberg and Lee (2017) presented SHAP (SHapley Additive exPlanations), a framework that quantifies the precise positive or negative impact of each feature by assigning a significance score to each feature in order to explain a single model prediction. Muhammad et al. (2024) demonstrated how SHAP can help overcome the black-box problem and make complex model conclusions interpretable by applying it to a deep learning model that classifies stock market patterns and using it to explain which features drove each prediction. However, great classification accuracy does not always translate into profitable investing strategies because the study focuses on trend-pattern categorisation rather than realistic trading profitability. Additionally, SHAP outputs are technical values that are difficult for a non-technical user to comprehend. Plainstock bridges this gap by turning SHAP feature contributions into explanations in plain English, allowing a young investor to read the reasoning behind a suggestion and determine whether or not to believe it.

(184 words)

2.6 Conclusion

Both Plainstock's design and motive are supported by the material discussed in this chapter. Each of the current tools has a flaw: Bloomberg Terminal is pricey and full of jargon, Robinhood makes suggestions without providing an explanation, open-source trading algorithms need programming knowledge, and human advisors are expensive and not necessarily impartial. LSTM and Random Forest can be used for stock prediction while also setting reasonable expectations, as demonstrated by the machine learning literature (Fischer and Krauss, 2018; Nelson et al., 2017; Breiman, 2001). This project's own conclusion that neither model outperformed a basic baseline supports this claim.

The use of indicators like RSI, MACD, and Bollinger Bands as model features is supported by the technical indicator literature (Wilder, 1978; Bollinger, 2001; Nelson et al., 2017). Lastly, the explainability literature demonstrates that SHAP can produce interpretable model predictions (Lundberg and Lee, 2017; Muhammad et al., 2024). Combining prediction, technical-indicator characteristics, and SHAP explanations into a free program that converts results into understandable English for non-technical users is the main gap that no reviewed effort addresses. Plainstock is intended to bridge this gap.

(182 words)

(Total: 1,263 words)

CHAPTER 3:PROJECT DESIGN

3.1 Domain and Users

Plainstock is a web application-based data science and machine learning effort in the retail investment technology (fintech) space. The initiative is intended for non-technical, first-time investors, usually young adults between the ages of 18 and 30, who wish to invest but do not have the funds or financial expertise to use professional tools. These people are unfamiliar with Github's open-source trading bots and command-line tools, which call for coding expertise. Additionally, they have trouble with programs that provide a result without any justification. In order for people to comprehend and evaluate the advice themselves, they require a straightforward instrument that provides a clear suggestion and explains, in basic English, why a stock should be bought or sold.

(117 words)

3.2 Justification of Design Choices

The system explains its recommendations in plain English rather than only showing a BUY or SELL result or raw technical numbers. This is because the target users are non-technical, and a plain-English explanation, together with a graph, helps them understand the reasoning behind a recommendation and decide for themselves whether to act on it.

To bridge the gap between technical and non-technical users, the project is supplied as a web application. Non-technical individuals can simply view a website without the need to install software or utilise a command line. Additionally, it neatly shows a price graph, the recommendation, and an explanation in plain English all in one location.

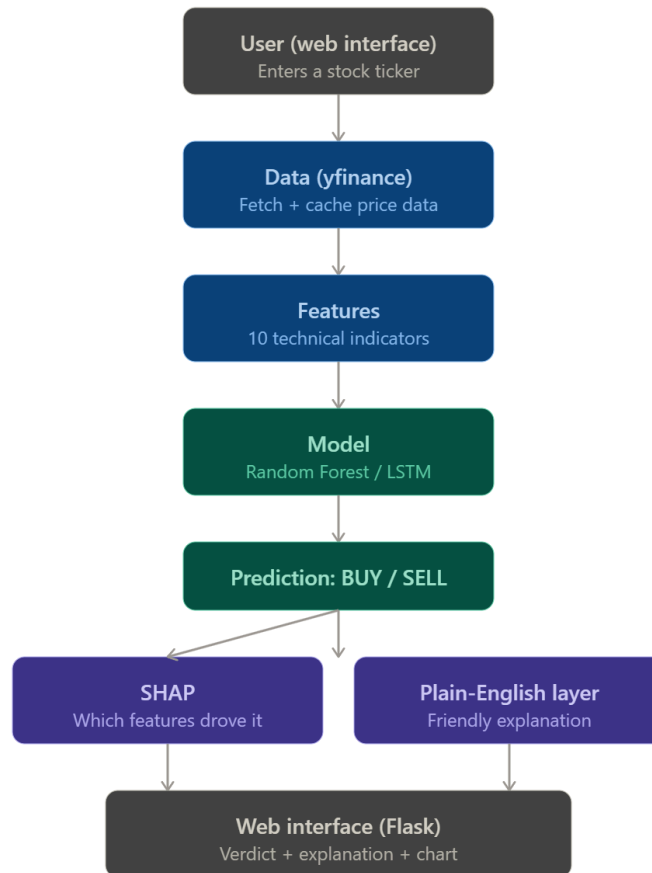
Due to the high cost of the current alternatives, the tool is free. The annual cost of a Bloomberg Terminal is around \$31,980. Human financial counsellors are extremely expensive and could be swayed by commission or emotion. The financial barrier for young, inexperienced investors who cannot afford these options is eliminated by making Plainstock free.

To allow for comparison, two models were employed: LSTM as a more sophisticated model and Random Forest as a basic baseline. The objective was to determine which model outperforms the others in terms of technical indicator analysis and BUY/SELL prediction, with the stronger model being designated as the primary model.

SHAP was included to explain which technical indicators pushed the model's decision. It assigns each feature a signed value positive means it pushed the model toward BUY, negative toward SELL. Importantly, SHAP does not make the prediction itself; it only explains why the model decided to buy or sell, adding a layer of transparency on top of the plain-English explanation.

(275 Words)

3.3 Overall Structure (Architecture)



As seen in Figure 1, the architecture is based on a pipeline. The system receives the input and retrieves price information from Yfinance when a user inputs a stock ticker they are interested in on the online interface. The system calculates the technical indicators using this data. These indicators are then used by the model to predict whether to buy or sell. The plain-English layer converts the indicator readings into a user-friendly recommendation and explanation, while SHAP separately identifies which features drove the model's decision. Lastly, a stock chart, the recommendation, and the description are shown on the online interface.

(100 words)

3.4 Technologies and Methods

Python was chosen as the main language because most machine learning libraries are written in Python, which makes it suitable for a project built around ML models. The remaining technologies were each chosen for a specific role in the pipeline.

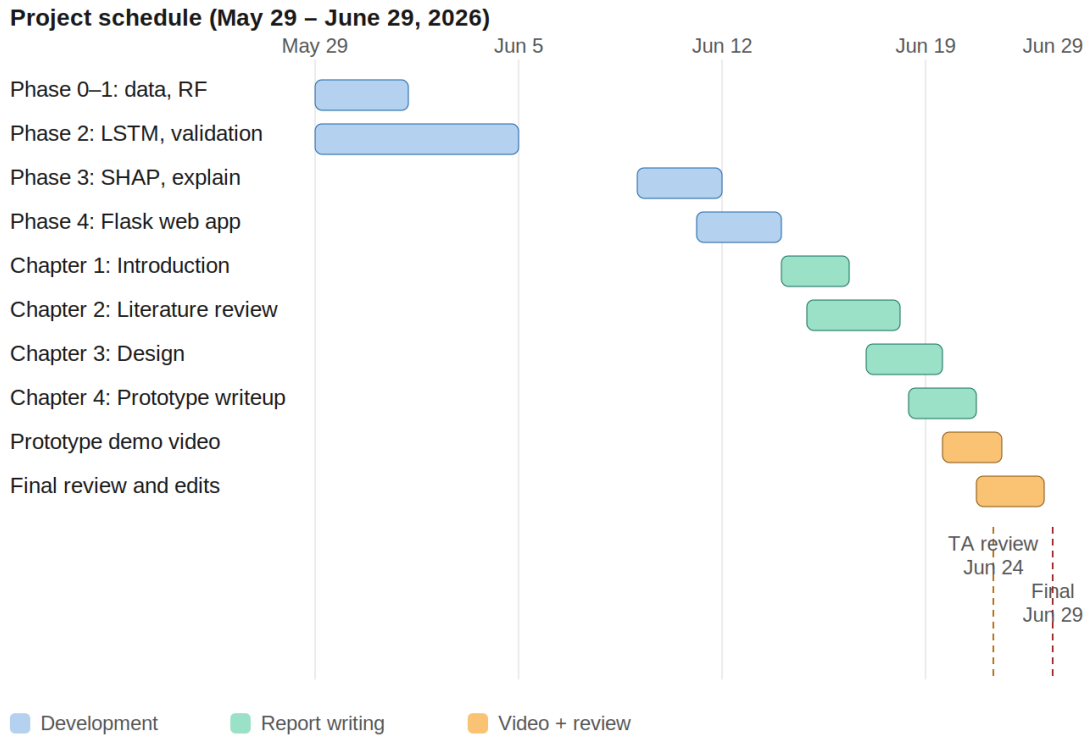
With a straightforward Python interface, yfinance offers free historical stock price data that is appropriate for a free tool. The pricing data is handled and processed by pandas, which is also used to compute the technical indicators. Since the indicators were computed directly in pandas instead of utilising a pre-built library, the underlying math is understood and was manually validated during development. A common, well-documented machine learning library, scikit-learn offers the Random Forest model. The LSTM model is constructed using TensorFlow/Keras because it supports sequential data and neural networks. The features that influenced the model's predictions are explained using SHAP.

Because Flask is a lightweight Python web framework, it was utilised for the online application. Without the additional complexity of a larger framework like Django, it is straightforward and ideal for a little project like this. The stock price chart was created using Plotly so that the user could observe how the price changed over time and quickly determine whether it had been rising or falling.

Pytest was used to create and execute the unit tests that verify every component of the system functions properly, and Git was used for version control to monitor changes throughout the project.

(239 words)

3.5 Work Plan



As seen in Figure 2, the project was developed in stages. Phase 1 produced the data fetcher, the initial indicator for the BUY/SELL goal, and the Random Forest baseline. Phase 0 established the project scaffold. The LSTM model, walk-forward validation, and the entire indicator set were added in Phase 2, followed by a fair comparison of the two models. The plain-English explanation layer (the rule-based plain-English generator and SHAP) was included in Phase 3. And Phase 4 developed the frontend and Flask web application. The pipeline expanded logically from data to the web interface as each stage built upon the one before it. The time left over after development was finished was used to make a demonstration video and write a preliminary report. The TA review on June 24 and the preliminary submission on June 29 are the two important deadlines.

(138 words)

3.6 Test and Evaluation Plan

Software testing is used to verify that the code functions properly, and model evaluation is used to gauge how accurate the predictions are.

To ensure that there are no bugs before a user uses the tool, a comprehensive set of unit tests covering the indicators, the BUY/SELL target, the models, and the web layer were built using Pytest for software testing. In order to prevent the model from inadvertently being trained on future data, leakage protections (such as chronological train/test divides and fitting the scaler on the training data only) were incorporated and the indicator calculations were manually checked against worked instances.

For model evaluation, the key aim is to test whether the model beats a majority baseline, and whether the trading strategy beats a simple buy-and-hold approach. This is evaluated using walk-forward validation across multiple folds, comparing accuracy against the majority baseline, and running a backtest against buy-and-hold with transaction costs included. The backtest is measured using financial metrics such as total return, Sharpe ratio, and maximum drawdown.

(171 words)

(Total: 1,040 words)

CHAPTER 4:FEATURE PROTOTYPE

4.1 Describing the prototype

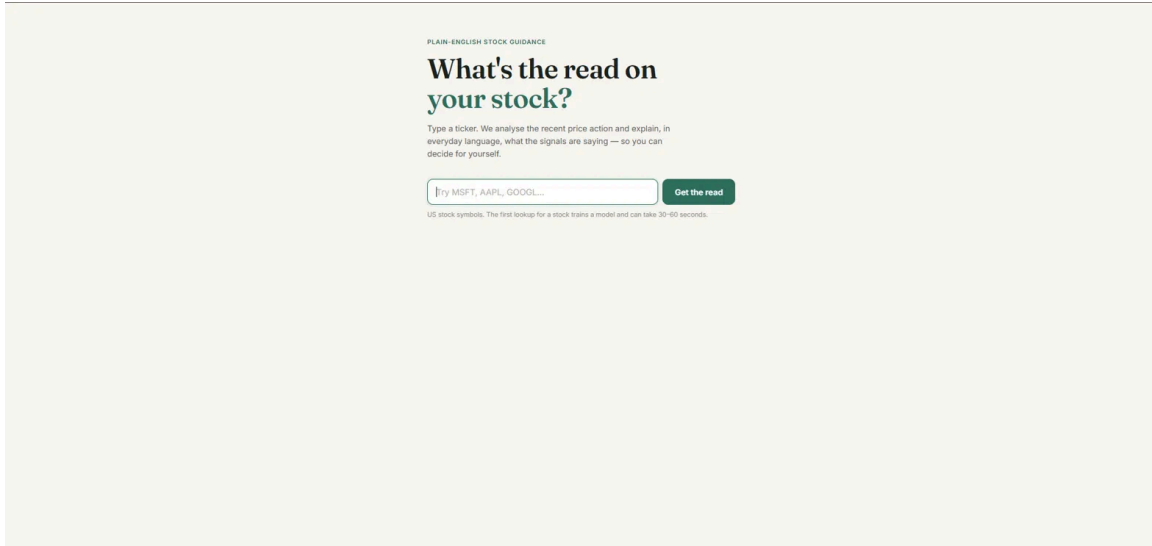
The complete web application is the feature prototype. When a user inputs a stock ticker for a firm they are interested in learning more about, the system provides a price chart, a plain-English explanation, and a recommendation (BUY or SELL).

This feature was chosen as the prototype because it is the core of the whole project; it brings all the main parts together in one flow. When the user enters a ticker, the system fetches data from yfinance, computes the technical indicators, passes them to the model to predict BUY or SELL, generates the explanation, and displays the recommendation, the explanation, and the chart on the web interface. Demonstrating this end-to-end flow shows that the central idea of the project is feasible: a non-technical user can get a data-driven recommendation with a clear explanation, all in one place.

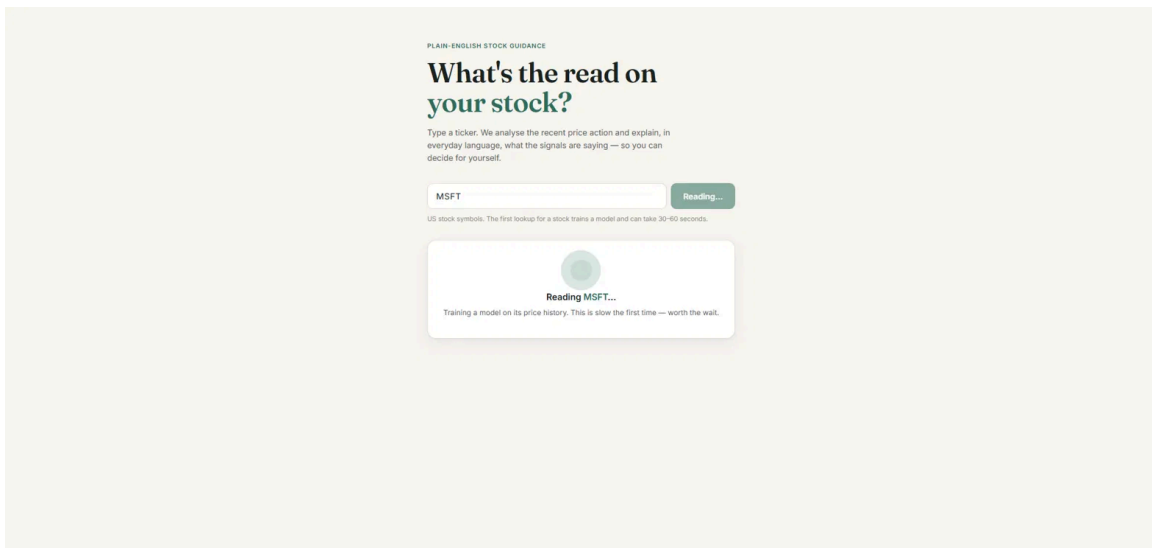
(138 words)

4.2 Describing the prototype

The user sees a simple page with a single search box and a brief description of the tool's functions when they launch the web application.

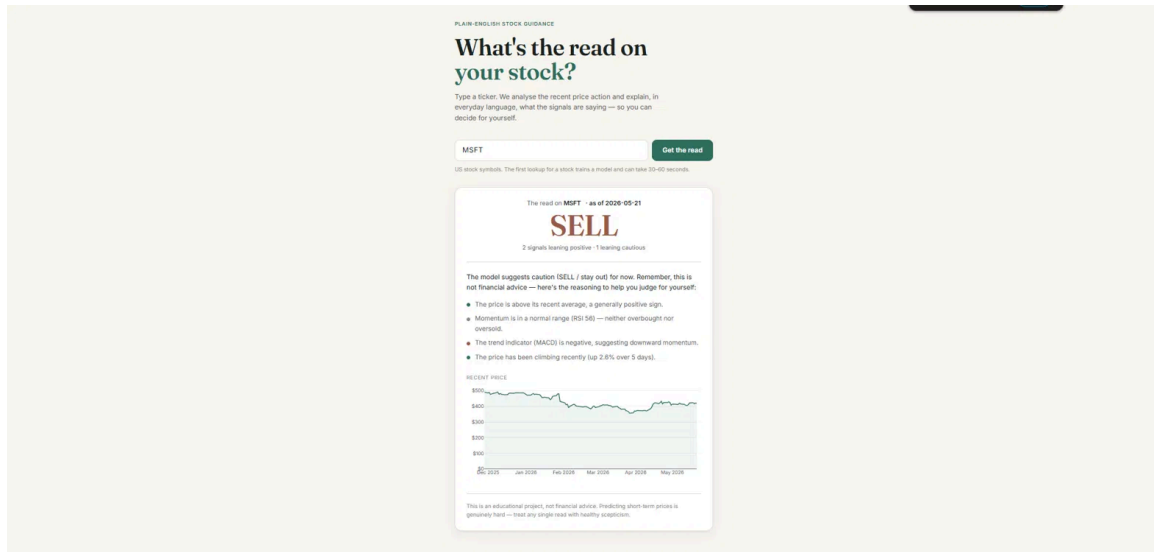


Click "Get the read" after entering a stock ticker, like MSFT. A loading state is displayed while the system operates, indicating that it may take 30 to 60 seconds for the initial lookup to train a model.



The request is sent to the backend in the background, where it retrieves pricing information from Yfinance and calculates the technical indicators. The LSTM model receives these indicators and makes a BUY or SELL prediction. The indicator readings are converted into simple sentences by the plain-English layer, which then returns the output to the page.

The result card:



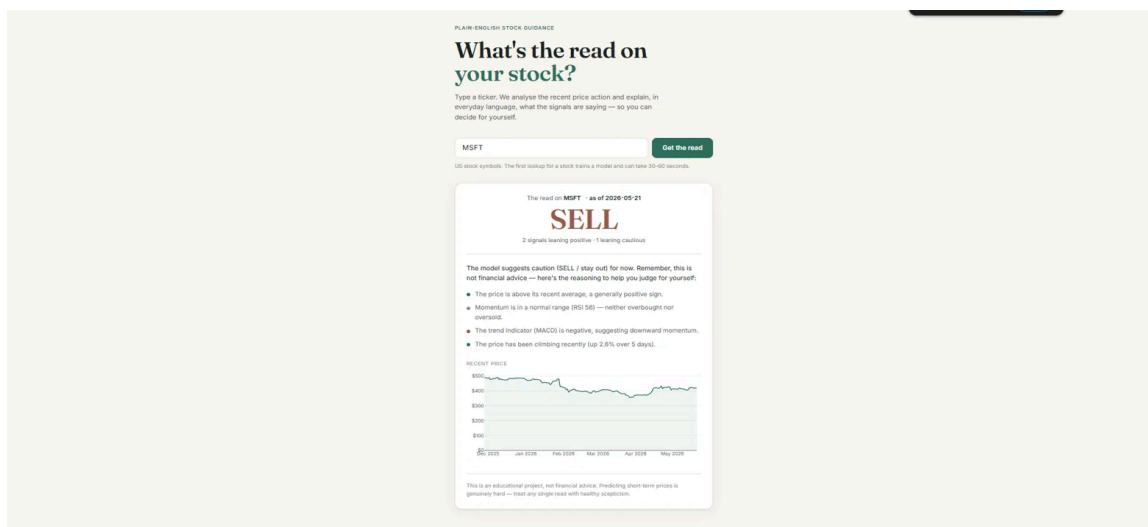
displays a current price chart, a plain-English explanation in bullet points, a brief overview of how many indications lean positive or cautious, and the decision (such as SELL) as a huge heading. The user is reminded with a disclaimer that this is not financial advice but rather an educational endeavour. This provides the user with all the information they require, including recommendations, explanations, and charts.

(185 words)

4.3 Evaluation

As a complete system, the prototype performs admirably. It completes the entire process without pausing, and it displays the outcome in an easy-to-understand interface that combines the pricing chart, the recommendation, and the explanation in plain English. This demonstrates that the project's main concept is workable.

However, building the prototype also revealed some limitations. First, the explanation does not always match the recommendation. This happens because the verdict and the explanation come from two different layers. The LSTM model makes the BUY or SELL decision by learning patterns from the technical indicators, while the plain-English layer describes the indicator values one by one in simple sentences. Because these are two separate layers, they can point in different directions. For example, the model may output SELL while several of the individual indicators still look positive, as seen in the MSFT example.



Second, the model's predictions are not reliably accurate. As shown in the design evaluation, the LSTM reached around 0.552 accuracy, which did not beat the majority baseline of 0.611. This means the recommendation should be treated with caution, which is why the interface includes a disclaimer that the tool is educational and not financial advice. Third, the first lookup for a stock is slow, because the model is trained on demand and can take 30–60 seconds, which affects the user experience. **(223 words)**

4.4 Improvements

One planned improvement is to try a hybrid CNN-LSTM model. This idea came from my supervisor's feedback that Random Forest and LSTM were not performing well enough, and from reading Sachdeva et al. (2025), who propose combining Convolutional Neural Networks with LSTM for stock trend prediction. In this hybrid approach, the CNN layers extract local patterns and important features from the indicator data, and the LSTM then models the longer-term dependencies to predict the trend. Testing whether this increases accuracy over the existing LSTM would be the goal. A hybrid model is a plausible next step to study rather than a surefire strategy to beat the baseline, but this should be addressed realistically: short-term price direction is actually difficult to anticipate, and much of the larger literature reports accuracy only in the mid-50% range.

The second improvement is making the description align with the recommendation. Currently, there may be a discrepancy between the explanation and the verdict, which could confuse the user. Instead of discussing the indications separately, the solution is to have the explanation represent what the model really utilised to make its choice. For instance, by applying SHAP to the same model that generates the verdict, the reasoning presented to the user fits the advice.

The third improvement is increasing the speed of the initial lookup, a less significant improvement. The model is currently trained on demand, which can take 30 to 60 seconds. This wait would be eliminated and the user experience would be enhanced by pre-training and caching models for popular stocks.

(256 words)

(Total: 802 words)

CHAPTER 5:REFERENCES

- Appel, G. (2005). *Technical analysis: Power tools for active investors*. Financial Times Prentice Hall.
- Bollinger, J. (2001). *Bollinger on Bollinger Bands*. McGraw-Hill.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669.
- Godel Discount. (2026). *Bloomberg Terminal cost 2026: \$31,980/year (full breakdown)*. <https://godeldiscount.com/blog/bloomberg-terminal-cost-2026>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30* (pp. 4765–4774).
- Money. (2021). *Robinhood and free trades: What every investor needs to know*. <https://money.com/robinhood-free-trades/>
- Muhammad, D., Ahmed, I., Naveed, K., & Bendeache, M. (2024). An explainable deep learning approach for stock market trend prediction. *Heliyon*, 10(21), Article e40095.
- Nelson, D. M. Q., Pereira, A. C. M., & de Oliveira, R. A. (2017). Stock market's price movement prediction with LSTM neural networks. In *2017 International Joint Conference on Neural Networks (IJCNN)* (pp. 1419–1426).
- Robinhood. (2026). *Commission-free stock trading & investing app*. <https://robinhood.com/us/en/>
- Sachdeva, V., Bolimela, A., Goyal, M. K., Kasireddy, L. C., Sholapurapu, P. K., Dahiya, A., & Goyal, K. (2025). Deep learning algorithms for stock market trend prediction in financial risk management. *Revista Latinoamericana de la Papa*, 29(1), 202–219.

Wilder, J. W. (1978). *New concepts in technical trading systems*. Trend Research.